# T3000 API & Protocol

Centralized Database Design · built Nov 13, 2026

# Centralized Database Design

> Backend configuration, encryption, schema management, and API reference. See centralized-database-multipc.md for the multi-PC architecture. See multipc-data-flow-details.md for per-table data flows.

## 1. Overview

The centralized database feature allows T3000 to store device data in a shared database server (MSSQL, PostgreSQL, or MySQL) instead of — or in addition to — the default local SQLite. This enables multi-PC deployments where all PCs see the same data.

### Supported Backends

| Backend | Library | Status |
|---|---|---|
| SQLite | SeaORM + sqlx-sqlite | Default, always available |
| MSSQL | tiberius 0.12 + bb8 pool | Primary target for enterprise |
| PostgreSQL | SeaORM + sqlx-postgres | Supported |
| MySQL | SeaORM + sqlx-mysql | Supported |

## 2. Password Encryption

Connection passwords are encrypted at rest using AES-256-GCM.

### Implementation

```
File: api/src/database_management/db_backend_config.rs
Functions: encrypt_password(), decrypt_password()
```

- **Algorithm:** AES-256-GCM (authenticated encryption)

- **Key:** Derived from machine-specific or hardcoded key (see `ENCRYPTION_KEY`)
- **Nonce:** 12-byte random, prepended to ciphertext
- **Storage:** Base64-encoded `nonce || ciphertext || tag` in `password_encrypted` column

### Flow

```
TEXT

Save config:  plaintext → encrypt_password() → base64 → DB_BACKEND_CONFIG.password_encry
Load config:  DB_BACKEND_CONFIG.password_encrypted → base64 decode → decrypt_password()
Connect:      plaintext password → database driver connection string
```

---

## 3. Schema Management

### SQL Dialect Files

Each supported backend has its own CREATE TABLE schema:

| File | Backend | Location |
|---|---|---|
| `webview_t3_device_schema.sql` | SQLite | `api/migration/sql/` |
| `webview_t3_device_mssql.sql` | MSSQL | `api/migration/sql/` |
| `webview_t3_device_postgres.sql` | PostgreSQL | `api/migration/sql/` |
| `webview_t3_device_mysql.sql` | MySQL | `api/migration/sql/` |

### Dialect Differences

| Feature | SQLite | MSSQL | PostgreSQL | MySQL |
|---|---|---|---|---|
| Auto-increment PK | `INTEGER PRIMARY KEY` | `INT IDENTITY(1,1)` | `SERIAL` | `INT AUTO_INCREMENT` |
| Text type | `TEXT` | `NVARCHAR(MAX)` | `TEXT` | `TEXT` |
| Boolean | `INTEGER` (0/1) | `BIT` | `BOOLEAN` | `TINYINT(1)` |
| Blob | `BLOB` | `VARBINARY(MAX)` | `BYTEA` | `LONGBLOB` |
| IF NOT EXISTS | Supported | Not supported (use `IF NOT EXISTS` wrapper) | Supported | Supported |

### Migration Strategy

On first connection to a central DB:

1. Check if tables exist (query `information_schema.tables` or equivalent)
2. Run the appropriate dialect's CREATE TABLE statements
3. Store `database.version` in APPLICATION_CONFIG (local)
4. On subsequent connects, compare version and run ALTER TABLE if needed

---

## 4. MSSQL Raw SQL Adapter

For MSSQL, we use `tiberius` directly instead of SeaORM (which doesn't support MSSQL natively).

### Connection Pool

RUST

```rust
// Type aliases in api/src/database_management/mssql_queries.rs
pub type MssqlPool = bb8::Pool<bb8_tiberius::ConnectionManager>;

// Pool created during startup with:
// - max_size: 10 (configurable)
// - connection_timeout: 30s
// - idle_timeout: 300s
```

### Query Pattern

RUST

```rust
pub async fn mssql_query_all(
    pool: &MssqlPool,
    table: &str,
    device_instance: Option<i64>,
) -> Result<Vec<HashMap<String, serde_json::Value>>, String> {
    let mut conn = pool.get().await?;
    let query = format!("SELECT * FROM [{}] WHERE ...", table);
    let stream = conn.query(&query, &[]).await?;
    // Convert tiberius rows to HashMap<String, Value>
    // ...
}
```

### MSSQL-Specific Functions

| Function | Purpose |
|---|---|
| `mssql_query_all` | SELECT * from table with optional filters |
| `mssql_query_by_id` | SELECT by primary key |
| `mssql_insert` | INSERT single row |
| `mssql_update` | UPDATE by primary key |
| `mssql_delete` | DELETE by primary key |
| `mssql_upsert` | INSERT or UPDATE (MERGE) |
| `mssql_count` | COUNT(*) from table |
| `mssql_create_tables` | Run schema migration |

## 5. DeviceDbConn Enum

Routes that access device data use `DeviceDbConn` to abstract over the active backend:

RUST

```rust
pub enum DeviceDbConn {
    SeaOrm(Arc<Mutex<DatabaseConnection>>),  // SQLite, Postgres, MySQL
    Mssql(MssqlPool),                        // MSSQL via tiberius
}
```

### Resolution

RUST

```rust
pub fn resolve_device_db_conn(state: &T3AppState) -> DeviceDbConn {
    if let Some(pool) = &state.mssql_pool {
        DeviceDbConn::Mssql(pool.clone())
    } else if let Some(conn) = &state.t3_device_conn {
        DeviceDbConn::SeaOrm(conn.clone())
    } else {
        // Fallback to local
        DeviceDbConn::SeaOrm(state.local_config_conn.clone().unwrap())
    }
}
```

## 6. REST API Endpoints

### Database Backend Configuration

| Method | Path | Purpose |
|---|---|---|
| `GET` | `/api/database/backend/config` | Get active backend config (password masked) |
| `POST` | `/api/database/backend/config` | Save/update backend config |
| `POST` | `/api/database/backend/test` | Test connection to a backend |
| `POST` | `/api/database/backend/activate` | Switch active backend |
| `GET` | `/api/database/backend/status` | Current connection status |
| `GET` | `/api/database/backend/tables` | List tables in active backend |

### INI Configuration (to be added)

| Method | Path | Purpose |
|---|---|---|
| `GET` | `/api/database/backend/ini` | Read current setting.ini [CentralDatabase] |
| `POST` | `/api/database/backend/ini` | Update setting.ini (local access only) |

### Request/Response Examples

#### POST /api/database/backend/config

JSON

```
{
  "backend_type": "mssql",
  "host": "192.168.1.100",
  "port": 1433,
  "database_name": "T3000_Central",
  "username": "t3000_user",
  "password": "secret123",
  "use_ssl": false,
  "role": "main",
  "store_logs": true
}
```

#### GET /api/database/backend/config (response)

JSON

```
{
  "id": 1,
  "backend_type": "mssql",
  "host": "192.168.1.100",
  "port": 1433,
  "database_name": "T3000_Central",
  "username": "t3000_user",
  "password": "********",
  "use_ssl": false,
  "is_active": true,
  "role": "main",
  "store_logs": true,
  "created_at": "2025-01-15T10:30:00Z",
  "updated_at": "2025-01-15T10:30:00Z"
}
```

---

## 7. DB_BACKEND_CONFIG Table Schema

SQL

```
CREATE TABLE IF NOT EXISTS DB_BACKEND_CONFIG (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    backend_type    TEXT NOT NULL DEFAULT 'sqlite',
    host            TEXT DEFAULT '',
    port            INTEGER DEFAULT 0,
    database_name   TEXT DEFAULT '',
    username        TEXT DEFAULT '',
    password_encrypted TEXT DEFAULT '',
    use_ssl         INTEGER DEFAULT 0,
    is_active       INTEGER DEFAULT 0,
    role            TEXT DEFAULT 'reader',
    store_logs      INTEGER DEFAULT 0,
    created_at      TEXT DEFAULT (datetime('now')),
    updated_at      TEXT DEFAULT (datetime('now'))
);
```

> This table lives in **local SQLite only** (Category G). It contains the encrypted credentials needed to connect to the central DB.

## 8. T3AppState Fields

RUST

```rust
pub struct T3AppState {
    // Primary webview database (users, auth)
    pub conn: Arc<Mutex<DatabaseConnection>>,

    // Device data connection — points to central DB when enabled, else local SQLite
    pub t3_device_conn: Option<Arc<Mutex<DatabaseConnection>>>,

    // Always points to local SQLite for Category G tables
    pub local_config_conn: Option<Arc<Mutex<DatabaseConnection>>>,

    // MSSQL connection pool (only set when active backend is MSSQL)
    pub mssql_pool: Option<MssqlPool>,

    // Multi-PC configuration (from setting.ini)
    pub central_db_enabled: bool,       // [CentralDatabase] enabled=1
    pub central_db_role: String,        // "main" or "reader"
    pub store_logs_to_central: bool,    // store_logs=1
}
```

## 9. Frontend: DatabaseConfigPage

**Location:** `src/t3-react/features/system/pages/DatabaseConfigPage.tsx`

### Sections

1. **Status Card** — Shows current backend type, connection status, table count
2. **Backend Selector** — Radio buttons: SQLite, PostgreSQL, MySQL, MSSQL
3. **Connection Form** — Host, port, database, username, password, SSL toggle
4. **Role Selector** (new) — Main / Reader radio buttons
5. **Store Logs Toggle** (new) — Checkbox to enable SYSTEM_LOGS writing
6. **Actions** — Test Connection, Save, Scan Network (for auto-discovery)

### State Management

Uses Zustand store with:

- `selectedType` — current backend selection

- `configs` — map of backend configs keyed by type
- `connectionStatus` — test result
- `role` — "main" | "reader"
- `storeLogs` — boolean